

EasyFood - Atividade 5

Implementando autenticação com JWT

Nesta atividade, vamos evoluir a EasyFood adicionando autenticação de usuários.

Ao final, a aplicação permitirá cadastro, login, identificação do usuário autenticado e proteção do cadastro de restaurantes.

```
POST /auth/register
POST /auth/login
GET /auth/me
POST /restaurants -> protegido
```

Parte 1 - PREPARE

Deixe sua EasyFood no mesmo ponto da aula anterior

Antes de começar, confirme que a aplicação já utiliza Prisma + PostgreSQL e está organizada em camadas.

```
src/
├── database/
│   └── prisma.js
├── modules/
│   └── restaurants/
│       ├── restaurant.service.js
│       ├── restaurant.controller.js
│       └── restaurant.routes.js
└── app.js
```

Execute:

```
node server.js
```

Confirme que as duas rotas continuam funcionando:

```
GET /restaurants
POST /restaurants
```

Parte 2 - INSTALE AS DEPENDÊNCIAS

Dentro da pasta do projeto, execute:

```
npm install jsonwebtoken bcryptjs dotenv
```

Vamos utilizar:

```
jsonwebtoken -> criação e validação do JWT
bcryptjs      -> hash e validação das senhas
dotenv        -> leitura das variáveis do arquivo .env
```

Parte 3 - CONFIGURE O JWT

1. Configure o arquivo .env

Abra o arquivo .env e adicione:

```
JWT_SECRET="troque-por-uma-chave-longa-e-aleatoria"
```

Seu arquivo deverá possuir pelo menos:

```
DATABASE_URL="..."
JWT_SECRET="troque-por-uma-chave-longa-e-aleatoria"
```

O JWT_SECRET deve ficar no .env. Não coloque a chave diretamente no código.

2. Carregue as variáveis de ambiente

Abra server.js e adicione na primeira linha:

```
require("dotenv").config();
```

O arquivo ficará:

```
require("dotenv").config();

const app = require("../src/app");

app.listen(3000, () => {
  console.log("EasyFood rodando na porta 3000");
});
```

Parte 4 - CRIE O USUÁRIO NO BANCO

3. Adicione o model User

Abra prisma/schema.prisma. Mantenha o model Restaurant e adicione:

```
model User {
  id      Int      @id @default(autoincrement())
  name    String   @db.VarChar(150)
  email   String   @unique @db.VarChar(150)
  password String   @db.VarChar(255)
}
```

Execute a migration:

```
npx prisma migrate dev --name criar-tabela-users
```

Parte 5 - CRIE O MÓDULO AUTH

Dentro de src/modules/, crie a pasta auth/. Ao final teremos:

```
src/
├── modules/
│   ├── restaurants/
│   └── auth/
│       ├── auth.service.js
│       ├── auth.controller.js
│       ├── auth.middleware.js
│       └── auth.routes.js
```

Parte 6 - AUTH SERVICE

4. Crie auth.service.js

```
src/modules/auth/auth.service.js
```

Adicione:

```
const jwt = require("jsonwebtoken");
const bcrypt = require("bcryptjs");
const prisma = require("../../database/prisma");
```

```

const JWT_SECRET = process.env.JWT_SECRET;

async function register({ name, email, password }) {
  const hash = await bcrypt.hash(password, 10);

  const user = await prisma.user.create({
    data: { name, email, password: hash }
  });

  return {
    id: user.id,
    name: user.name,
    email: user.email
  };
}

async function login({ email, password }) {
  const user = await prisma.user.findUnique({
    where: { email }
  });

  if (!user) {
    return null;
  }

  const senhaConfere = await bcrypt.compare(
    password,
    user.password
  );

  if (!senhaConfere) {
    return null;
  }

  const token = jwt.sign(
    {
      sub: user.id,
      email: user.email
    },
    JWT_SECRET,
    {
      expiresIn: "1d"
    }
  );

  return {
    token,
    user: {
      id: user.id,
      name: user.name,
      email: user.email
    }
  };
}

module.exports = {
  register,
  login
};

```

No cadastro, a senha passa pelo bcrypt antes de ser armazenada. No login, bcrypt.compare() confere a senha e jwt.sign() gera o token.

Parte 7 - AUTH CONTROLLER

5. Crie auth.controller.js

```
src/modules/auth/auth.controller.js
```

Adicione:

```
const authService = require("../auth.service");

async function register(req, res) {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
    return res.status(400).json({
      error: "Nome, e-mail e senha são obrigatórios"
    });
  }

  try {
    const user = await authService.register({
      name,
      email,
      password
    });

    res.status(201).json(user);
  } catch (error) {
    if (error.code === "P2002") {
      return res.status(409).json({
        error: "E-mail já cadastrado"
      });
    }

    res.status(500).json({
      error: "Erro interno do servidor"
    });
  }
}

async function login(req, res) {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({
      error: "E-mail e senha são obrigatórios"
    });
  }

  try {
    const result = await authService.login({
      email,
      password
    });

    if (!result) {
      return res.status(401).json({
        error: "Credenciais inválidas"
      });
    }

    res.json(result);
  } catch (error) {
    res.status(500).json({
      error: "Erro interno do servidor"
    });
  }
}
```

```

    res.status(500).json({
      error: "Erro interno do servidor"
    });
  }
}

module.exports = {
  register,
  login
};

```

Parte 8 - AUTH MIDDLEWARE

6. Crie auth.middleware.js

```
src/modules/auth/auth.middleware.js
```

Adicione:

```

const jwt = require("jsonwebtoken");

const JWT_SECRET = process.env.JWT_SECRET;

function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;

  if (
    !authHeader ||
    !authHeader.startsWith("Bearer ")
  ) {
    return res.status(401).json({
      error: "Token não fornecido"
    });
  }

  const token = authHeader.split(" ")[1];

  try {
    const payload = jwt.verify(
      token,
      JWT_SECRET
    );

    req.user = {
      id: payload.sub,
      email: payload.email
    };

    next();
  } catch (error) {
    return res.status(401).json({
      error: "Token inválido ou expirado"
    });
  }
}

module.exports = authenticate;

```

O middleware verifica o header Authorization, valida o token e, se estiver correto, chama next().

Parte 9 - AUTH ROUTES

7. Crie auth.routes.js

```
src/modules/auth/auth.routes.js
```

Adicione:

```
const express = require("express");

const controller = require("../auth.controller");
const authenticate = require("../auth.middleware");

const router = express.Router();

router.post("/register", controller.register);
router.post("/login", controller.login);

router.get("/me", authenticate, (req, res) => {
  res.json({
    message: "Você está autenticado!",
    user: req.user
  });
});

module.exports = router;
```

Parte 10 - ADICIONE AUTH AO APP

8. Atualize src/app.js

Importe as rotas:

```
const authRoutes =
  require("../modules/auth/auth.routes");
```

Registre:

```
app.use("/auth", authRoutes);
```

O arquivo ficará semelhante a:

```
const express = require("express");
const cors = require("cors");
const path = require("path");

const restaurantRoutes =
  require("../modules/restaurants/restaurant.routes");

const authRoutes =
  require("../modules/auth/auth.routes");

const app = express();

app.use(cors());
app.use(express.json());

app.use(
  express.static(
    path.join(__dirname, "../public")
  )
);

app.use("/auth", authRoutes);
```

```
app.use("/restaurants", restaurantRoutes);

module.exports = app;
```

Parte 11 - TESTE O CADASTRO

Inicie a aplicação:

```
node server.js
```

Faça:

```
POST http://localhost:3000/auth/register
```

Body:

```
{
  "name": "Aluno",
  "email": "aluno@easyfood.com",
  "password": "123456"
}
```

Resultado esperado: 201 Created.

```
{
  "id": 1,
  "name": "Aluno",
  "email": "aluno@easyfood.com"
}
```

A senha não deve aparecer na resposta.

Parte 12 - TESTE O LOGIN

```
POST http://localhost:3000/auth/login
```

Body:

```
{
  "email": "aluno@easyfood.com",
  "password": "123456"
}
```

A resposta deverá possuir um token:

```
{
  "token": "...",
  "user": {
    "id": 1,
    "name": "Aluno",
    "email": "aluno@easyfood.com"
  }
}
```

Copie o valor do token.

Parte 13 - TESTE /auth/me

Faça:

```
GET http://localhost:3000/auth/me
```

Sem token, o resultado esperado é:

```
401 Unauthorized
```

Agora envie o header:

```
Authorization: Bearer COLE_SEU_TOKEN_AQUI
```

Faça novamente a requisição. Resultado esperado:

```
200 OK
```

Parte 14 - PROTEJA O POST /restaurants

9. Atualize restaurant.routes.js

Abra src/modules/restaurants/restaurant.routes.js e importe:

```
const authenticate =  
  require("../auth/auth.middleware");
```

Altere o POST:

```
router.post(  
  "/",  
  authenticate,  
  controller.create  
);
```

O arquivo deverá ficar:

```
const express = require("express");  
  
const controller =  
  require("../restaurant.controller");  
  
const authenticate =  
  require("../auth/auth.middleware");  
  
const router = express.Router();  
  
router.get("/", controller.list);  
  
router.post(  
  "/",  
  authenticate,  
  controller.create  
);  
  
module.exports = router;
```

GET /restaurants continua público. POST /restaurants passa a exigir um JWT válido.

Parte 15 - TESTE FINAL

1. Tente cadastrar um restaurante sem token:

```
POST /restaurants
```

Resultado esperado:

```
401 Unauthorized
```

2. Faça a mesma requisição enviando:

```
Authorization: Bearer SEU_TOKEN
```

Resultado esperado:

```
201 Created
```

Fluxo final

```
CADASTRO
  ↓
senha -> bcrypt -> hash -> PostgreSQL
  ↓
LOGIN
  ↓
bcrypt.compare()
  ↓
JWT
  ↓
Authorization: Bearer <token>
  ↓
MIDDLEWARE
  ↓
ROTA PROTEGIDA
```

Estrutura final

```
easy-food/
├── prisma/
│   ├── migrations/
│   └── schema.prisma
├── src/
│   ├── database/
│   │   └── prisma.js
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.service.js
│   │   │   ├── auth.controller.js
│   │   │   ├── auth.middleware.js
│   │   │   └── auth.routes.js
│   │   └── restaurants/
│   │       ├── restaurant.service.js
│   │       ├── restaurant.controller.js
│   │       └── restaurant.routes.js
│   └── app.js
├── .env
├── server.js
└── package.json
```

Atividade concluída quando

- ☐ O usuário consegue se cadastrar.
- ☐ A senha é armazenada como hash.
- ☐ O usuário consegue fazer login.
- ☐ O login devolve um JWT.
- ☐ /auth/me funciona apenas com token.
- ☐ GET /restaurants continua público.
- ☐ POST /restaurants retorna 401 sem token.
- ☐ POST /restaurants funciona com token válido.